C++ Chapter 2: Getting Started

Contents

C++ Chapter 2: Getting Started 1
How Popular is C++? 1
Read: Think C++ 1
Do: Online Tutorials 1
Introduction to C++ 2
What Is a Program? 3
Values and Variables 4
Constants 4
Operators and Operands 4
Expressions 6
Input - Process - Output 6
Input 6
Tutorial 1: Favorite Number 7
Tutorial 2: Wages 8
Assignment 1: Miles to Kilometers 10
Assignment Submission 12

Time required: 90 minutes

How Popular is C++?

https://pypl.github.io/PYPL.html

https://www.tiobe.com/tiobe-index/

Read: Think C++

- Chapter 2 Variables and Types

Do: Online Tutorials

- C++ Tutorial

- C++ Intro
- C++ Syntax
- C++ Output (Print Text)
- C++ New Lines
- C++ Comments
- C++ Variables
    - C++ Declare Multiple Variables
    - C++ Identifiers
    - C++ Constants
- C++ User Input
- C++ Data Types
    - C++ Numbers
    - C++ Booleans
    - C++ Characters
    - C++ Strings
- C++ Operators
    - C++ Assignment Operators
    - C++ Comparison Operators
    - C++ Logical Operators

Introduction to C++

C++ (pronounced see plus plus) is a general-purpose programming language that supports both procedural and object-oriented programming paradigms. It is compiled into a binary executable. It is regarded as an intermediate-level language, as it comprises both high-level and low-level language features. Key features include strong typing, manual memory management, and support for classes and objects.

Machine Independent but Platform Dependent: A C++ executable is not platform-independent (compiled programs on Linux won't run on Windows), however they are machine independent.

A C++ program can be compiled and executed on any operating system that has a C++ compiler.

C++ is one of the most popular programming languages and is implemented on a wide variety of hardware and operating system platforms. As an efficient performance driven programming language, it is used in systems software, application software, device drivers, embedded software, high-performance server and client applications, and entertainment software such as video games.

What is C++?

C++ is a cross-platform language that can be used to create high-performance applications.

Who developed C++?

Bjarne Stroustrup developed C++ (originally named "C with Classes") in 1983 at Bell Labs as an enhancement to the C programming language.

What Is a Program?

A program is a sequence of instructions that specifies how to perform a computation. The computation might be something mathematical, like solving a system of equations or finding the roots of a polynomial. It can also be a symbolic computation, like searching and replacing text in a document or (strangely enough) compiling a program.

The instructions (or commands, or statements) look different in different programming languages, but there are a few basic functions that appear in just about every language:

> **input:** Get data from the keyboard, or a file, or some other device.
>
> **output:** Display data on the screen or send data to a file or other device.
>
> **math:** Perform basic mathematical operations like addition and multiplication.
>
> **testing:** Check for certain conditions and execute the appropriate statements.
>
> **repetition:** Perform some action repeatedly, usually with some variation.

Believe it or not, that's pretty much all there is to it. Every program you've ever used, no matter how complicated, is made up of functions that look like these. One way to describe programming is the process of breaking a large, complex task up into smaller and smaller

subtasks until eventually the subtasks are simple enough to be performed with one of these simple functions.

Values and Variables

The number four (4) is an example of a numeric value. In mathematics, 4 is an integer value. Integers are whole numbers, which means they have no fractional parts, and an integer can be positive, negative, or zero. Examples of integers include 4, -19, 0, and -1005. In contrast, 4.5 is not an integer, since it is not a whole number.

Constants

C++ supports constants. Constants are declared like variables with the addition of the **const** keyword:

```
const double PI{3.14159265358979};
```

This is a double data type representation of PI in C++ as a constant. C++ does not have a built-in constant for PI.

Once declared and initialized, a constant can be used like a variable in all but one way—a constant may not be reassigned. It is illegal for a constant to appear on the left side of the assignment operator (=) outside its declaration statement. A subsequent statement like

```
PI = 2.5;
```

would cause the compiler to issue an error message.

Operators and Operands

Symbol	Operation	Example	Description
+	Addition	a + b	Adds two numbers.
–	Subtraction	a – b	Subtracts one number from another.
-	Negation	- a	Change sign of operand.
*	Multiplication	a * b	Multiplies one number by another.

/	Division	a / b	Divides one number by another and gives the result as a floating-point number. If both values are int, the result is integer division.
%	Remainder or Modulus	a % b	Divides one number by another and gives the remainder.

Operators are special symbols that represent computations like addition and multiplication. The values the operator is applied to are called operands.

Variables in C++ must be defined before they can be used.

```
int number;
double a_double{24.0};
float b_double{60.0};
```

The following table lists some of the data types, sizes, and ranges in C++.

Type	Size (in bytes)	Range
short int	2	-32,768 to 32,767
unsigned short int	2	0 to 65,535
unsigned int	4	0 to 4,294,967,295
int	4	-2,147,483,648 to 2,147,483,647
long int	4	-2,147,483,648 to 2,147,483,647
unsigned long int	4	0 to 4,294,967,295
signed char	1	-128 to 127
unsigned char	1	0 to 255
char	1	-128 to 127
float	4	7 decimal points
double	8	15 decimal points
long double	16	30
bool	1	

Expressions

- **statement**: a unit of code that does something – a basic building block of a program.
- **expression:** a statement that has a value – for instance, a number, a string, the sum of two numbers, etc. 4+2, x-1, and "Hello, world!\n" are all expressions.

Input - Process - Output

All programs have three basic components.

Input

User-entered values are passed to the input stream using the **cin** keyword which takes input from the keyboard using the operator **>>**

```
/**
 * Filename: temperature_converter.cpp
 * Written by:
 * Written on:
 * Description: Convert fahrenheit to celsius
 */

#include <iostream>
int main()
{
    double fahrenheit;
    double celsius;
    std::cout << "+-----------------------------------+" << std::endl;
    std::cout << "|-- Convert Fahrenheit to Celsius --|" << std::endl;
    std::cout << "+-----------------------------------+" << std::endl;
    std::cout << " Enter temperature in F: ";
    // Program pauses until user enters a value and presses Enter
    // User input is assigned to fahrenheit
    std::cin >> fahrenheit;
    // Convert fahrenheit to Celsius
    celsius = ((fahrenheit - 32) * 5.0) / 9.0;
    // Output results to the console
    std::cout << fahrenheit << " Fahrenheit " =
              << celsius << " Celsius" << std::endl;
    // Return success status to the OS
    return 0;
}
```

Example run:

```
Enter temperature in degrees F: -40
-40 Fahrenheit = -40 Celsius
```

Tutorial 1: Favorite Number

Create a C++ program named **favorite_number.cpp**

```cpp
/**
 * @name favorite_number.cpp
 * @date
 * @brief Get name and favorite number from user
 */
#include <iostream>

int main()
{
    // Declare variable
    int favorite_number;
    std::string name;
    // Prompt the user for input
    std::cout << "\nHello, what is your name?: ";
    // Get input from keyboard
    std::cin >> name;
    // Prompt the user for a number
    std::cout << "\nEnter your favorite number between 1 and 100: ";
    // Get input from keyboard
    std::cin >> favorite_number;
    // Output to console, split long line of code
    std::cout << "Amazing " << name << "! "
              << favorite_number << " is my favorite number too.\n";
    return 0;
}
```

Example run:

```
Hello, what is your name?: Bill

Enter your favorite number between 1 and 100: 42
Amazing Bill! 42 is my favorite number too.
```

Tutorial 2: Wages

```cpp
/**
 * Filename: wages.cpp
 * Written by:
 * Written on:
 * Description: Calculate wages per month
 */

#include <iostream>

int main()
{
    // Declare variables to store hourly wage, hours worked, monthly salary
    double hourlyWage;
    double hoursWorked;
    double monthlySalary;

    // Ask the user to input their hourly wage
    std::cout << "Enter your hourly wage: $";
    // Store the user's input in 'hourlyWage'
    std::cin >> hourlyWage;

    // Ask the user to input the number of hours worked
    std::cout << "Enter the number of hours worked in a month: ";
    // Store the user's input in 'hoursWorked'
    std::cin >> hoursWorked;

    // Calculate the monthly salary by multiplying wage and hours
    monthlySalary = hourlyWage * hoursWorked;

    // Display the result to the user
    std::cout << "Your monthly salary is: $" << monthlySalary << std::endl;

    // Return 0 to indicate successful program execution
    return 0;
}
```

Example run:

```
Enter your hourly wage: $75
Enter the number of hours worked in a month: 40
Your monthly salary is: $3000
```

Assignment 1: Miles to Kilometers

Convert the following program from Java to C++.

```java
/*******************************************************
 * Name: MilesToKM.java
 * Written by:
 * Written on:
 * Purpose: Convert Miles to Kilometers from user input
 *******************************************************/

// Import Scanner library for input
import java.util.Scanner;

public class MilesToKM {
    Run | Debug
    public static void main(String[] args) {
        // Declare Scanner object and initialize with
        // predefined standard input object, System.in
        Scanner keyboard = new Scanner(System.in);

        // Constant for converting Miles to KM
        final double KM_PER_MILE = 1.609344;

        // Variables for input and calculations
        double miles;
        double kilometers;

        // Prompt the user for input
        System.out.println("+-----------------------------------+");
        System.out.println("|--  Convert Miles to Kilometers  --|");
        System.out.println("+-----------------------------------+");
        System.out.print("Enter the distance in miles: ");

        // Get double from the keyboard, assign double to variable
        miles = keyboard.nextDouble();

        // Convert miles to kilometers
        kilometers = miles * KM_PER_MILE;

        // Round kilometers to 2 decimal places
        kilometers = Math.round(kilometers * 100.0) / 100.0;

        // Display answer to user
        System.out.println(miles + " miles = " + kilometers + " kilometers");
        // Close scanner object
        keyboard.close();
    }
}
```

Example run:

```
+-----------------------------------+
|--  Convert Miles to Kilometers  --|
+-----------------------------------+
Enter the distance in miles: 10
10.0 miles = 16.09 kilometers
```

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.